



Progettazione di applicazioni web full stack

Autenticazione e autorizzazione in NestJS



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia



- Una libreria/middleware che **NestJS** usa per gestire l'autenticazione degli utenti in modo modulare e riusabile
 - NestJS organizza l'applicazione
 - Passport si occupa di verificare *chi è l'utente* e gestire lo stato dell'applicazione in caso di utente autenticato
- **Passport** gestisce l'autenticazione utente attraverso diverse *strategie*
 - **local**, con email/username e password
 - **jwt**, attraverso l'uso di token (*JSON Web Tokens*)



- Con **Passport** in **NestJS** è possibile:
 - definire *come* un utente si autentica
 - proteggere rotte attraverso le **guard**
 - validare credenziali (email/username e password) o token
 - allegare l'utente autenticato alle request HTTP

Pacchetti da installare



```
npm install @nestjs/passport passport passport-local
npm install @nestjs/jwt passport-jwt
npm install bcrypt
npm install -D @types/passport-jwt @types/bcrypt @types/passport-local
```

Package	Utilizzo
<code>passport</code>	Core engine di autenticazione
<code>passport-local</code>	Strategia per l'implementazione della login con credenziali
<code>passport-jwt</code>	Strategia per implementare l'autenticazione con token JWT
<code>@nestjs/passport</code>	Integrazione ufficiale con NestJS di guardie e strategia
<code>@nestjs/jwt</code>	Integrazione ufficiale con NestJS della gestione dei token
<code>@types/...</code>	Supporto alla gestione dei tipi TS (solo development, -D)
<code>bcrypt</code>	Package per la gestione di contenuti criptati

Moduli e dipendenze (I)



```
npx nx g @nx/nest:library libs/server/auth --controller --service --importPath=@server/auth
npx nx g @nx/nest:library libs/server/security --importPath=@server/security
```

- Disponiamo già di una libreria **NestJS** per la gestione degli utenti nel database PostgreSQL (**users**)
- Definiamo una libreria **NestJS** separata per la gestione dell'autenticazione, con *strategie* e *guards* (**auth**)
 - l'utente viene autenticato con email/username e password (strategia *passport-local*)
 - a valle dell'autenticazione, viene emesso un *token JWT* da includere nell'*authorization header* delle successive richieste HTTP
- Le rotte sono protette da accessi indesiderati posizionandole dietro *guards* che verificano la validità del token nella richiesta (in un'ulteriore libreria **security**, per evitare dipendenze circolari)

Moduli e dipendenze (II)



Evoluzione della libreria @server/users (I)



File `user.entity.ts`

- Il campo `email` verrà utilizzato per l'autenticazione
- Nel database la `password` verrà salvata in formato criptato

```
@Entity('users')
export class UserEntity {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({type:'varchar', length: 255, nullable: false})
  name: string;

  @Column({type:'varchar', length:320, nullable: false, unique: true})
  email: string;

  @Column()
  passwordHash: string;

  @Column({
    type: 'enum',
    enum: UserRole,
    default: UserRole.USER
  })
  role: UserRole;
}
```

Evoluzione della libreria @server/users (II)



File `create-user.dto.ts`

- Vengono imposte nuove regole di validazione su `email` e `password`

```
export class CreateUserDto {
  @IsString()
  @NotEmpty()
  name: string;

  @IsEmail()
  @NotEmpty()
  email: string;

  @IsString()
  @NotEmpty()
  @MinLength(8)
  @Matches(/[A-Z]/, {
    message: 'Password must contain at least one uppercase letter' })
  @Matches(/[?!#@]/, {
    message: 'Password must contain at least one symbol among ? ^ ! # @' })
  // @Matches(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[?!#@]).{8,}$/), {
  //   message:
  //     'Password troppo debole: min 8 caratteri, maiuscola, minuscola, numero e simbolo (? ^ ! #
  // })
  password: string;
```


Evoluzione della libreria @server/users (III)



File `users.repository.ts` e `users.service.ts`

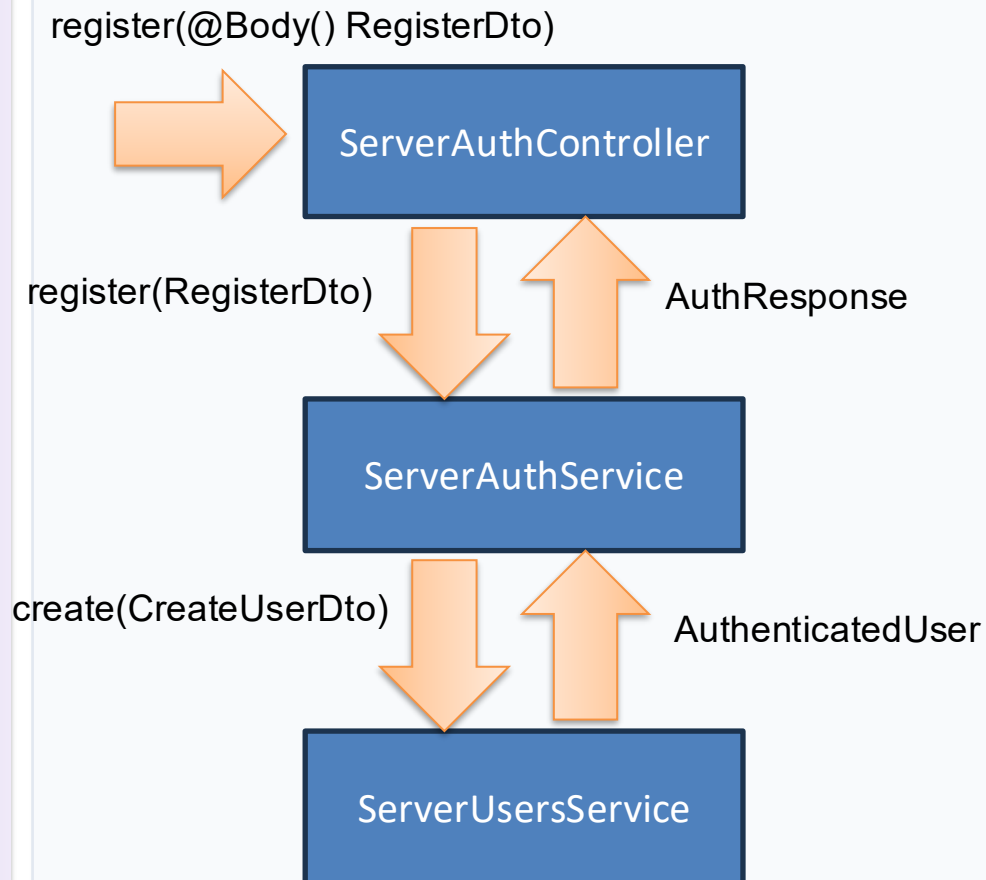
- Viene modificato il metodo `createOne` in `users.repository.ts` per inserire la versione criptata della password

```
async createOne(dto: CreateUserDto, passwordHash: string): Promise<UserEntity> {  
  const user = this.repository.create({  
    name: dto.name,  
    email: dto.email,  
    passwordHash: passwordHash,  
    role: dto.role  
  });  
  return this.repository.save(user);  
}
```

- Il metodo `create` in `users.service.ts` viene modificato di conseguenza

```
async create(dto: CreateUserDto): Promise<UserEntity> {  
  const existing = dto.email  
    ? await this.usersRepository.findByEmail(dto.email)  
    : null;  
  
  if (existing) {  
    throw new ConflictException('Email already in use');  
  }  
  
  const passwordHash = await bcrypt.hash(dto.password, 10);  
  return this.usersRepository.createOne(dto, passwordHash);  
}
```

La libreria @server/auth (II) – Registrazione



La libreria @server/auth (II) – Registrazione



register(@Body() RegisterDto)



ServerAuthController

```
import { CreateUserDto } from "@server/users";  
export class RegisterDto extends CreateUserDto {}
```

```
re @Post('register')  
  @ApiBody({  
    schema: {  
      type: 'object',  
      properties: {  
        name: { type: 'string', example: 'Devis' },  
        email: { type: 'string', example: 'devis.bianchini@unibs.it' },  
        password: { type: 'string', example: 'Password1!' },  
        role: { type: 'string', enum: Object.values(UserRole), example: UserRole.USER },  
      },  
      required: ['name', 'email', 'password', 'role'],  
    },  
  })  
  register(@Body(ValidationPipe) dto: RegisterDto) {  
    return this.serverAuthService.register(dto);  
  }
```

La libreria @server/auth (II) – Registrazione



register(@Body() RegisterDto)



ServerAuthController

register(RegisterDto)



ServerAuthService

```
import { UserRole } from "@server/users";

export interface AuthenticatedUser {
  id: number;
  email: string;
  role: UserRole;
  name: string;
}
```

```
import { AuthenticatedUser } from "../authenticated-user.interface";

export class AuthResponse {
  access_token: string;
  user: AuthenticatedUser;
}
```

```
async register(dto: RegisterDto): Promise<AuthResponse> {
  const user = await this.usersService.create(dto);

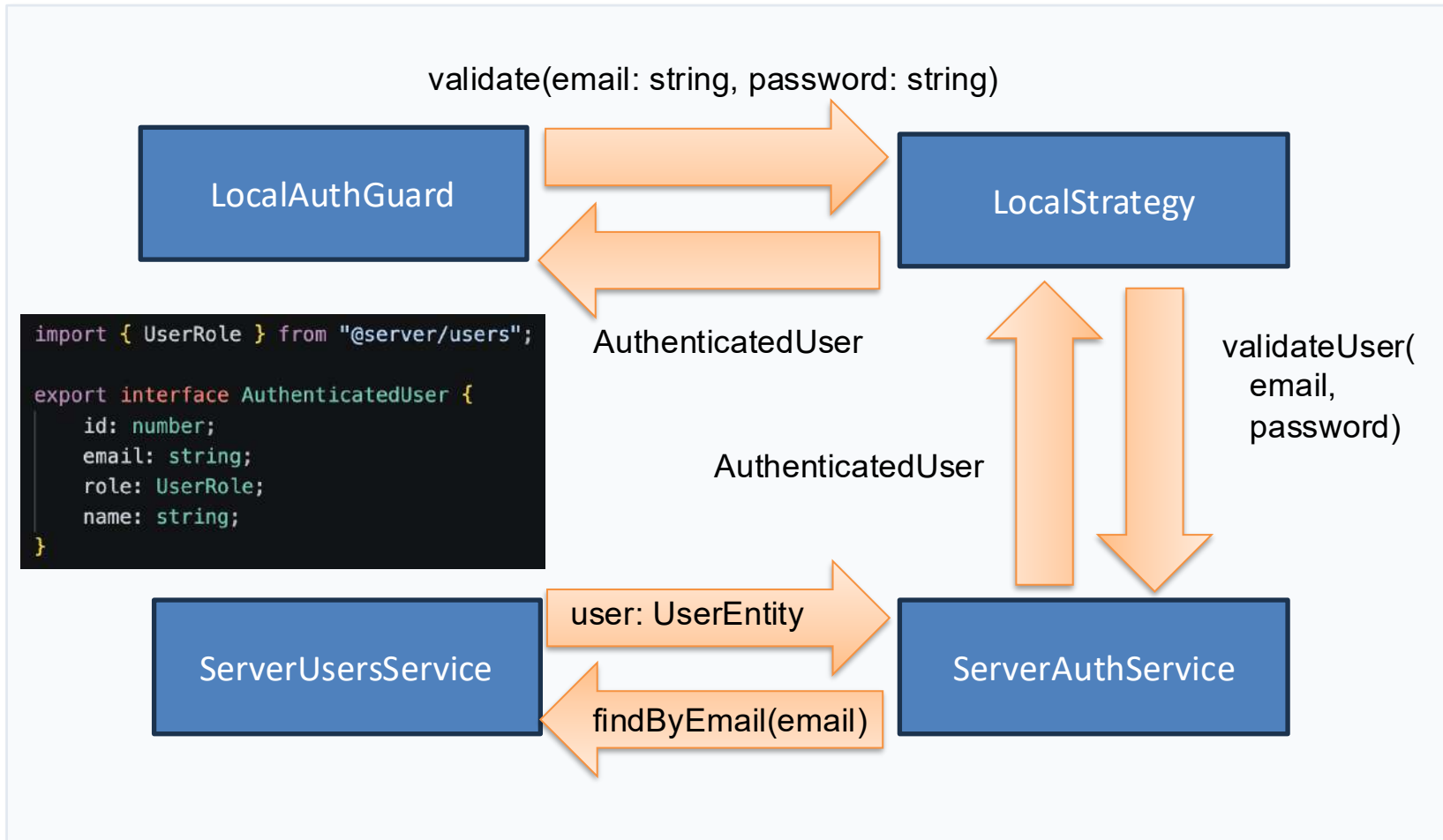
  const { passwordHash, ...result } = user;
  return this.login(result);
}
```

Definire una strategia



- Una strategia estende **PassportStrategy** (nel pacchetto **@nestjs/passport**)
 - Richiede un insieme di *options*, da passare alla super-classe nel costruttore tramite il metodo `super()`
 - Richiede l'implementazione della funzione **validate**, onde specificare come verificare i dati inseriti dall'utente (email/username e password, token JWT) e restituire l'utente autenticato o un'eccezione in caso di errore di autenticazione

La libreria @server/auth - Credenziali



Passport-local strategy (I)



File `local.strategy.ts` nella libreria `Auth`

- Verifica le credenziali dell'utente
- Il suo metodo `validate` viene invocato quando la richiesta HTTP attraversa la corrispondente guardia (sulle rotte)
- Si appoggia su `ServerAuthService` (per aumentare la modularità)

```
import { Injectable } from "@nestjs/common";
import { Strategy } from 'passport-local';
import { PassportStrategy } from "@nestjs/passport";
import { ServerAuthService } from "../auth.service";
import { AuthenticatedUser } from "../interfaces/authenticated-user.interface";

@Injectable()
export class LocalStrategy extends PassportStrategy(Strategy) {
  constructor(private authService: ServerAuthService) {
    super({usernameField: 'email'});
  }

  async validate(email: string, password: string): Promise<AuthenticatedUser> {
    return this.authService.validateUser(email, password);
  }
}
```

- Di default questa strategia funziona con i campi `username` e `password`
- È possibile cambiare specificando opportunamente le `options`

Passport-local strategy (II)



File `auth.service.ts`

- Si appoggia al modulo `users` (e relativo servizio) per cercare l'utente nel database, al fine di verificarne le credenziali
- Ritorna un `result` che è compatibile con `AuthenticatedUser`

```
import { UserRole } from "@server/users";

export interface AuthenticatedUser {
  id: number;
  email: string;
  role: UserRole;
  name: string;
}
```

```
@Injectable()
export class ServerAuthService {
  // Injecting the used services
  constructor(private readonly usersService: ServerUsersService) {}

  async validateUser(email: string, password: string): Promise<AuthenticatedUser> {
    const user = await this.usersService.findByEmail(email);

    if(!user) throw new NotFoundException("Credentials not valid");

    const passwordMatches = await bcrypt.compare(password, user.passwordHash);
    if(!passwordMatches) {
      throw new UnauthorizedException("Credentials not valid!");
    }

    const { passwordHash, ...result } = user;
    return result;
  }
}
```


Auth controller (I)



La **local guard** va messa prima del login nel file **auth.controller.ts**

- Intercetta la rotta per effettuare l'autenticazione con credenziali
- Attiva la corrispondente strategia attraverso **LocalAuthGuard**
- Una volta verificate le credenziali, viene eseguito il metodo login del servizio, che farà partire la *strategia JWT*

- Wrapper class per **AuthGuard('local')** per impedire la proliferazione di «magic strings»

- Il servizio **ServerAuthService** genera il token JWT con un *payload*
- È necessario iniettare **jwtService** (package **@nestjs/jwt**)

```
@ApiTags('Auth APIs')
@Controller('auth')
export class ServerAuthController {
  constructor(private serverAuthService: ServerAuthService) {}

  @UseGuards(LocalAuthGuard)
  @Post('login')
  @ApiBody({
    schema: {
      type: 'object',
      properties: {
        email: { type: 'string', example: 'devis.bianchini@unibs.it' },
        password: { type: 'string', example: 'Password1!' },
      },
      required: ['email', 'password'],
    },
  })
  login(@Request() req: RequestWithUser) {
    return this.serverAuthService.login(req.user);
  }
}
```

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
```

```
@Injectable()
export class LocalAuthGuard extends AuthGuard('local') {}
```

```
constructor(private readonly usersService: ServerUsersService,
  private readonly jwtService: JwtService) {}
```

```
async login(user: AuthenticatedUser): Promise<AuthResponse> {
  const payload = {sub: user.id, name: user.name, email: user.email, role: user.role};
  return {access_token: await this.jwtService.signAsync(payload), user}
}
```

La libreria @server/auth (II) – JWT strategy



La **JWT guard** proteggerà qualsiasi rotta invocando la JWT strategy, che verifica la presenza di un JWT token valido

- Il costruttore verifica il token contenuto nella **richiesta HTTP** in arrivo
- Per farlo, utilizza una **chiave segreta** salvata sul server per evitare manomissioni
- Il metodo **validate** verifica il payload contenuto nel token e emette un'istanza di **AuthenticatedUser**

```
import { UserRole } from "@server/users";

export interface JwtPayload {
  sub: number;
  email: string;
  role: UserRole;
  name: string;
}
```

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { AuthenticatedUser } from '../interfaces/authenticated-user.interface';
import { JwtPayload } from '../interfaces/jwt-payload.interface';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    const secret = process.env.SECRET_KEY;

    if (!secret) {
      throw new Error('SECRET_KEY is not defined');
    }

    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: secret,
    });
  }

  validate(payload: JwtPayload): AuthenticatedUser {
    return {
      id: payload.sub,
      email: payload.email,
      role: payload.role,
      name: payload.name
    };
  }
}
```

La libreria @server/auth (II) – JWT strategy



File `jwt.strategy.ts`

- Il metodo `validate` viene invocato quando la richiesta HTTP attraversa `AuthGuard('jwt')`
- La guard viene posizionata a protezione di tutte le rotte dell'applicazione che devono essere preservate da accessi indesiderati

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { AuthenticatedUser } from '../interfaces/authenticated-user.interface';
import { JwtPayload } from '../interfaces/jwt-payload.interface';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    const secret = process.env.SECRET_KEY;

    if (!secret) {
      throw new Error('SECRET_KEY is not defined');
    }

    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: secret,
    });
  }

  validate(payload: JwtPayload): AuthenticatedUser {
    return {
      id: payload.sub,
      email: payload.email,
      role: payload.role,
      name: payload.name
    };
  }
}
```

La libreria @server/auth (II) – JWT strategy



File `jwt.strategy.ts`

- Deve essere specificata una stringa (qui chiamata `SECRET_KEY`) che serve per verificare lato server il token una volta estratto dalla richiesta HTTP

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { AuthenticatedUser } from '../interfaces/authenticated-user.interface';
import { JwtPayload } from '../interfaces/jwt-payload.interface';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    const secret = process.env.SECRET_KEY;

    if (!secret) {
      throw new Error('SECRET_KEY is not defined');
    }

    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: secret,
    });
  }

  validate(payload: JwtPayload): AuthenticatedUser {
    return {
      id: payload.sub,
      email: payload.email,
      role: payload.role,
      name: payload.name
    };
  }
}
```

La libreria @server/auth (II) – JWT strategy



File `jwt.strategy.ts`

- Il token JWT viene estratto quando questa strategia viene istanziata, viene verificato se il token è scaduto (`ignoreExpiration: true`) e viene utilizzata la chiave segreta a questo scopo

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { AuthenticatedUser } from '../interfaces/authenticated-user.interface';
import { JwtPayload } from '../interfaces/jwt-payload.interface';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    const secret = process.env.SECRET_KEY;

    if (!secret) {
      throw new Error('SECRET_KEY is not defined');
    }

    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: secret,
    });
  }

  validate(payload: JwtPayload): AuthenticatedUser {
    return {
      id: payload.sub,
      email: payload.email,
      role: payload.role,
      name: payload.name
    };
  }
}
```

La libreria @server/auth (II) – JWT strategy



File `jwt.strategy.ts`

```
validate(payload: JwtPayload) {  
  return {  
    userId: payload.sub,  
    email: payload.email,  
    role: payload.role,  
    name: payload.name  
  };  
}
```

- Il metodo `validate` restituisce il payload contenuto nell'***authentication header*** (e creato nel metodo `login` del servizio di autenticazione)

Rendere tutto pienamente operativo (I)



File `auth.module.ts`

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { ServerUsersModule } from '@server/users';
import { ServerAuthController } from './auth.controller';
import { JwtStrategy } from './strategies/jwt.strategy';
import { ServerAuthService } from './auth.service';
import { LocalStrategy } from './strategies/local.strategy';
import 'dotenv/config';

@Module({
  imports: [
    ServerUsersModule,
    PassportModule,
    JwtModule.register({
      secret: process.env.SECRET_KEY,
      signOptions: { expiresIn: '24h' }
    })
  ],
  controllers: [ServerAuthController],
  providers: [ServerAuthService, LocalStrategy, JwtStrategy],
  exports: [ServerAuthService],
})
export class ServerAuthModule {}
```

Rendere tutto pienamente operativo (II)



File `index.ts`

```
export * from './lib/auth.module';
export * from './lib/auth.service';
export * from './lib/dto/login.dto';
export * from './lib/dto/register.dto';
export * from './lib/interfaces/authenticated-user.interface';
export * from './lib/interfaces/jwt-payload.interface';
```

Tipo	Injectable	Va in exports	Perché
Service	✓	✓	usato via DI
Repository	✓	a volte	data access
Strategy	✓	✗	usata da Passport
Guard	✓	✗	usata via decorator
DTO	✗	✗	non provider

Come cambiano le rotte protette (I)



File `users.controller.ts` nella libreria `users`

```
import { Body, Controller, Delete, Get, Param, Patch, Post, Query, ParseIntPipe, ParseEnumPipe, ValidationPipe, HttpException } from '@nestjs/common';
import { ServerUserService } from './users.service';
import { ApiTags, ApiBody, ApiQuery, ApiBearerAuth } from '@nestjs/swagger';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';
import { UserRole } from './dto/user-role.enum';
import { CurrentUser, JwtAuthGuard, Public, Roles, RolesGuard } from '@server/security';

@ApiTags('Users APIs')
@Controller('users')
export class ServerUsersController {
  constructor(private serverUsersService: ServerUserService) {}

  @Get() // GET /users or /users?role=value
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.ADMIN)
  @ApiBearerAuth()
  @ApiQuery({ name: 'role', required: false, enum: UserRole })
  getUsers(@Query('role', new ParseEnumPipe(UserRole, { optional: true })) role?: UserRole) {
    return this.serverUsersService.getUsers(role);
  }

  @Get('me')
  @UseGuards(JwtAuthGuard)
  @ApiBearerAuth()
  getMe(@CurrentUser() user: unknown) {
    return user;
  }
}
```

`@UseGuards ()`

`@Roles ()`

`@CurrentUser ()`

`@ApiBearerAuth ()`

La libreria @server/security (I)



File `jwt-auth.guard.ts`

- Wrapper class per `AuthGuard('jwt')` per impedire la proliferazione di «magic strings»

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') { }
```

La libreria @server/security (II)



File **role.guard.ts** implementa il meccanismo dell'*autorizzazione*

- Implementa **canActivate()**, che deve rispondere true o false (decide se una richiesta HTTP può procedere o meno)
- Da **ExecutionContext** (contesto della richiesta corrente) è possibile estrarre la rotta chiamata, l'utente che ha inviato la richiesta, altri metadati opportunamente definiti

```
import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { UserRole } from '@server/users';
import { ROLES_KEY } from '../decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<UserRole[]>(
      ROLES_KEY,
      [context.getHandler(), context.getClass()],
    );

    if (!requiredRoles || requiredRoles.length === 0) {
      return true;
    }

    const request = context.switchToHttp().getRequest();
    const user = request.user;

    if (!user) {
      throw new ForbiddenException('User not found in request');
    }

    if (!requiredRoles.includes(user.role)) {
      throw new ForbiddenException('Insufficient permissions');
    }

    return true;
  }
}
```

La libreria @server/security (II)



File **role.guard.ts** implementa il meccanismo dell'*autorizzazione*

- **Reflector** serve per recuperare i metadati associati a classi e metodi (per esempio, ciò che sta nei decorator)
- **context.getHandler()** restituisce il metodo del controller
- **context.getClass()** restituisce la classe controller
- **context.switchToHttp().getRequest()** ottiene la richiesta HTTP (dentro la quale abbiamo messo lo **user**)

```
import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { UserRole } from '@server/users';
import { ROLES_KEY } from '../decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<UserRole[]>(
      ROLES_KEY,
      [context.getHandler(), context.getClass()],
    );

    if (!requiredRoles || requiredRoles.length === 0) {
      return true;
    }

    const request = context.switchToHttp().getRequest();
    const user = request.user;

    if (!user) {
      throw new ForbiddenException('User not found in request');
    }

    if (!requiredRoles.includes(user.role)) {
      throw new ForbiddenException('Insufficient permissions');
    }

    return true;
  }
}
```

La libreria @server/security (III)



| *decorators* *role.decorator.ts* e *current-user-decorator.ts*

```
import { SetMetadata } from "@nestjs/common";
import { UserRole } from '@server/users';

export const ROLES_KEY = 'roles';
export const Roles = (...roles: UserRole[]) => SetMetadata(ROLES_KEY, roles);
```

```
import { createParamDecorator, ExecutionContext } from "@nestjs/common";

export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    return request.user;
  }
);
```

```
export * from './lib/decorators/current-user.decorator';
export * from './lib/decorators/roles.decorator';
export * from './lib/guards/jwt-auth.guard';
export * from './lib/guards/roles.guard';
```

Modifica della app che usa le librerie



File apps/api/src/main.ts

```
const config = new DocumentBuilder()
  .addBearerAuth()
  .setTitle('Nx-NestJS')
  .setDescription('API DOCUMENTATION')
  .setVersion('1.0')
  .addTag('api')
  .build();
```

Visualizzazione delle API in Swagger



Authorize

api

Example of API

GET

/api/data

Users APIs

GET

/api/users

POST

/api/users

GET

/api/users/me

GET

/api/users/interns

GET

/api/users/{id}

PATCH

/api/users/{id}

DELETE

/api/users/{id}



- Link alla documentazione di **Passport** con **NestJS**:
<https://docs.nestjs.com/recipes/passport>



Progettazione di applicazioni web full stack

Autenticazione e autorizzazione in NestJS



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia